COMP5331 Project Proposal

Beyond Random Anomalies: Evaluating Graph Anomaly Detection with Structure-Preserving Anomaly Injection

Group 5

September 2026

1 Project Information

1.1 Title

Beyond Random Anomalies: Evaluating Graph Anomaly Detection with Structure-Preserving Anomaly Injection

1.2 Type of Project

Research

1.3 Group No.

Group 5

2 Group Members

Member 1

- **Student ID:** 21359076
- **Student name:** ZHANG, Xingzhi
- **Research/FYP supervisor (if any):** WANG, Shuai
- **Own research/FYP topic (if any) and why this project is different:** My research topics are software security and computer system security. I mainly focus on compiler, CPU/GPU, operating system and their security issues.
- **Declaration:** I declare that this project is done solely within the scope of this course (COMP5331), and not within any other scope such as other courses or research projects.

Member 2

- **Student ID:** 20944721
- **Student name:** Liu,Minghao
- **Research/FYP supervisor (if any):** Yi R. (May) Fung
- **Own research/FYP topic (if any) and why this project is different:** My own research topic is post-training of large language models (LLMs), e.g., fine-tuning of pretrained language models. This project involves no language models, no post-training, and shares no data, code, or experiments with my research.
- **Declaration:** I declare that this project is done solely within the scope of this course (COMP5331), and not within any other scope such as other courses or research projects.

Member 3

- **Student ID:** 21268617
- **Student name:** JIANG Guanyu
- **Research/FYP supervisor (if any):** Yi R. (May) Fung
- **Own research/FYP topic (if any) and why this project is different:** My research focuses on building autonomous agentic systems driven by foundation models (LLMs / MLLMs). My goal is to ensure these agents operate effectively and reliably, and create positive real-world impact.
- **Declaration:** I declare that this project is done solely within the scope of this course (COMP5331), and not within any other scope such as other courses or research projects.

Member 4

- **Student ID:** 50032340
- **Student name:** YIN, Han
- **Research/FYP supervisor (if any):** YI, ke
- **Own research/FYP topic (if any) and why this project is different:** My research focuses on multi-agent system. My goal is to reduce the token cost for multi-agent multi-table question answering.
- **Declaration:** I declare that this project is done solely within the scope of this course (COMP5331), and not within any other scope such as other courses or research projects.

Member 5

- **Student ID:** 21274070
- **Student name:** ZHAO, Wenshuo
- **Research/FYP supervisor (if any):** HE, Junxian

- **Own research/FYP topic (if any) and why this project is different:** My research primarily focuses on the optimization of large language models — specifically, designing better pipelines for synthesizing training data and developing more effective algorithms to improve LLM performance. My research background differs considerably from this project, which centers on data mining and knowledge graphs and is therefore distinct from my area of expertise.
- **Declaration:** I declare that this project is done solely within the scope of this course (COMP5331), and not within any other scope such as other courses or research projects.

Member 6

- **Student ID:** 21349849
- **Student name:** WANG Chun
- **Research/FYP supervisor (if any):** Long Chen
- **Own research/FYP topic (if any) and why this project is different:** My research focuses on video generation, including the development and evaluation of generative models for synthesizing and editing video content. This project studies graph anomaly detection and uses no video data, models, or experiments from my research.
- **Declaration:** I declare that this project is done solely within the scope of this course (COMP5331), and not within any other scope such as other courses or research projects.

3 Brief Description

3.1 Introduction

Graph anomaly detection studies unusual nodes, edges, or subgraphs in relational data. It is useful in applications such as recommender systems, communication networks, financial transfers, and network security, where an unusual interaction may indicate fraud, abuse, or an unexpected event.

Reliable edge-level labels are rarely available in real graph streams. A common evaluation protocol therefore treats observed interactions as normal and injects synthetic edges as anomalies. In the simplest protocol, a timestamp and a valid pair of nodes are sampled and a previously nonexistent edge is inserted. This procedure creates labels at low cost, but it may also create anomalies that are easy to recognize because their structural or temporal context differs from that of ordinary interactions.

A recent study by Matthieu Latapy and Stephany Rajeh, *Trivial Graph Features and Classical Learning are Enough to Detect Random Anomalies* (TGF), reports that simple historical-graph features and a Random Forest can detect randomly injected anomalous edges effectively [1]. The released implementation represents a temporal stream as triples (t_i, u_i, v_i), constructs history graphs using either a fixed number of previous interactions (H-type) or a fixed time duration (G-type), and extracts degree, weighted-degree, edge-weight, rank, and history-graph statistics. This result provides a reproducible baseline for studying whether the measured performance depends on the random-injection protocol.

This project asks:

Do simple graph features remain effective when synthetic anomalies preserve the structural and temporal context of normal interactions?

We treat the anomaly generator as an experimental factor. The study first reproduces TGF on MovieLens, a bipartite user–item stream, and UCI Messages, a unipartite communication stream. It then replaces independent random sampling with controlled generators that match endpoint profiles, local structure, and temporal context. The primary comparison concerns how the generator changes measured detector performance; it does not require a new large prediction model. TGF is listed under Topic 1 of the COMP5331 suggested-paper page. Although the displayed topic name is "Clustering," the selected paper and this extension study graph mining and knowledge discovery on temporal graph streams.

3.2 Background and Motivation

TGF constructs a history graph for each arriving interaction, computes inexpensive graph features, and trains a Random Forest to distinguish observed links from injected links. Its released code provides preprocessing, anomaly injection, feature generation, and learning/testing examples for MovieLens and UCI Messages. These properties make TGF a suitable baseline for isolating the effect of anomaly construction.

Random injection can connect endpoints whose historical degree, activity, neighborhood, or recent timing is unlike that of an ordinary interaction. In a communication graph, it may create a link with no local context. In a recommender graph, it may pair a user with an item whose popularity is atypical for that user's history. A detector can exploit these differences even when they are artifacts of the benchmark rather than properties of the intended anomaly.

The central methodological question is therefore whether detector performance changes after these shortcuts are controlled. We do not assume that matching more variables automatically produces a realistic anomaly or a monotonically harder task. Each generator will be evaluated against the observed distribution of normal interactions. If a matching rule fails to reduce its intended discrepancy or introduces a substantial unrelated shift, we will report it as a distinct synthetic mechanism and qualify the corresponding conclusion.

This focus also determines how we use related methods. SedanSpot studies anomalous edges in streams using burst and connectivity information, while MIDAS targets sudden micro-clusters of related edges. SpotLight considers broader structural patterns in streaming graphs, and NetWalk and AddGraph represent learned dynamic-network approaches. These methods provide context for possible graph signals, but they do not define directly comparable baselines for the main single-edge experiment. We therefore use TGF as the controlled detector baseline and treat the other methods as background or optional extensions.

3.3 Research Questions

RQ1: How strongly does measured detection performance depend on the anomaly generator?
We compare random injection with endpoint-, local-structure-, and temporal-context-matched generators while fixing the dataset, chronological split, anomaly ratio, detector, and metrics.

RQ2: Which observable signals make a synthetic anomaly detectable?
We examine endpoint profiles, graph-type-specific local structure, and recent temporal activity through feature-group comparisons and generator diagnostics.

RQ3: Do detectors learn transferable anomaly signals or generator-specific shortcuts?
We compare same-generator training/testing with a train–test matrix whose rows and columns use different generators.

RQ4: Can a small set of interpretable features improve detection on harder synthetic anomalies?
If TGF features lose performance after matching, we evaluate local and temporal extensions without making a large neural model part of the required study.

3.4 Proposed Methodology

We first reproduce the official TGF pipeline on MovieLens and UCI Messages. We will pin the source-code commit, dataset versions, history parameters, injection ratio, random seeds, and classifier settings. The official implementation will be run unchanged for the reproduction. We will separately audit whether the current interaction enters the history before its features are computed and compare this behavior with the strict pre-arrival protocol used by the extension.

For the extension, an event at time t is generated and scored using only interactions with timestamps strictly less than t. All events sharing timestamp t use the same frozen history and update that history only after scoring. An injected edge must satisfy the graph-type constraints, contain no self-loop or duplicate, and be absent from the observed edge set used by the declared protocol. MovieLens candidates must respect the user–item partitions. The main extension will use pairs absent from the complete observed stream; a "not previously observed" definition will be treated as a separate sensitivity condition.

We implement four generators through a common interface:

G0: Random injection.
A valid timestamp and endpoint pair are sampled subject to the non-existence and graph-type constraints. This is the TGF control condition.

G1: Endpoint-matched injection.
Candidate endpoints are selected to match the historical degree, weighted degree, or activity rank of endpoints in observed interactions. The initial implementation will use auditable bins or standardized distances and will record insufficient candidate support.

G2: Local-structure-matched injection.
G1 constraints are retained and local context is matched. UCI Messages may use common-neighbor, Jaccard, Adamic–Adar, or related two-hop statistics on the unipartite history graph. MovieLens will use bipartite-compatible two-hop or co-interaction statistics.

G3: Temporal-context-matched injection.
G2 constraints are retained and recent activity, time since last endpoint activity, and active status within a declared history window are matched.

The labels G0–G3 describe intended controls, not a guaranteed difficulty ordering. For each injected edge, we will store its reference context, candidate-pool size, matching distance, and rejection reasons. We will retain the nested terminology only when diagnostics support it.

The matching procedure will be fitted from information available in the relevant training history. For example, degree bins, standardization constants, activity windows, and matching tolerances will be declared in the configuration and selected without looking at test labels. Candidate rejection will be counted by reason, including observed pairs, partition violations, self-loops, duplicates, and insufficient matching support. A generator that cannot produce the requested number of legal edges will be marked as support-limited rather than silently reducing the anomaly count.

An optional extension will study coordinated or micro-cluster anomalies, in which related anomalous edges appear within a short period. MIDAS is relevant to this setting [2], but the extension will remain separate because it changes the anomaly unit and requires a compatible evaluation protocol.

3.5 Evaluation and Analysis

For each dataset and generator, we evaluate the TGF Random Forest as the primary detector and Logistic Regression as a simple comparison. We compare the original TGF feature set with an extended set containing graph-specific local and temporal features. Feature transformations are fitted on the training period, hyperparameters are selected on validation data, and the test period is reserved for final evaluation.

We use chronological train, validation, and test periods, initially 60%/20%/20% by time. A timestamp is assigned to only one split. The main anomaly ratio is 5%, with 1% and 10% sensitivity settings. Each stochastic condition is repeated with at least five seeds. PR-AUC is the primary metric because anomalies are a minority class; ROC-AUC, Precision@K, and Recall@K are secondary metrics, with K equal to the number of test anomalies.

The main factorial comparison crosses the two datasets, four generators, two detectors, two feature sets, and five seeds at the 5% anomaly ratio. The extended experiments use the same data and generator caches for feature ablations and cross-generator testing. This design makes the generator the primary changing factor while keeping the detector and class prior aligned. The anomaly ratio, history window, and official-versus-causal protocol are varied only in declared sensitivity experiments.

For the primary comparisons, we will report mean and standard deviation across seeds and retain paired per-seed differences when the same split is used. We will not tune thresholds or select features from test performance. A failed or invalidated run will remain recorded and will be excluded from aggregation until it is rerun under the declared configuration.

Before a detector result is included in the analysis, we compare observed and injected edges using distribution plots, standardized mean differences, Kolmogorov–Smirnov statistics, Wasserstein distances, acceptance rates, and matching distances. These checks establish whether a generator reduces the discrepancy it is intended to control and whether it introduces other detectable shifts.

We will summarize stochastic results across seeds using means and standard deviations and will report paired per-seed differences when the same split and seed are shared by two generators. Precision@K and Recall@K are reported at a fixed test-set anomaly count rather than at a threshold chosen from the test labels. If bootstrap intervals or formal tests are used, the pairing and resampling procedure will be recorded with the corresponding table. These choices keep model

comparison separate from post-hoc threshold selection.

The main comparison is

$$\text{G0 Random} \rightarrow \text{G1 Endpoint} \rightarrow \text{G2 Local} \rightarrow \text{G3 Temporal}.$$

For each verified matching step, we compare PR-AUC and Recall@K with the corresponding control. Feature sufficiency and leave-one-group-out ablations measure the contributions of endpoint/TGF, local, and temporal features. The cross-generator experiment trains on each generator and tests on all four generators; a diagonal advantage or an off-diagonal drop would indicate generator-specific learning.

Every run is driven by a saved configuration containing the dataset checksum, code revision, generator parameters, seed, split boundaries, feature groups, classifier parameters, and history definition. We store generated streams, diagnostics, predictions, and metrics with run identifiers, so that all report tables and figures can be regenerated from machine-readable artifacts.

The main threats to validity are construct validity, temporal leakage, and limited external validity. A synthetic edge is a controlled label, not evidence of real fraud or attack. Timestamp-batched histories and the separation of official and pre-arrival protocols address leakage concerns. Conclusions will be stated separately for the bipartite and unipartite datasets, and a result will not be generalized beyond the tested graph types without additional evidence.

3.6 Expected Outcomes and Contribution

This project studies evaluation methodology rather than proposing a new anomaly-detection architecture. Simple features may remain effective after matching, which would support a broad interpretation of the TGF result. Performance may instead decrease after observable sampling differences are removed, indicating that random injection creates an optimistic benchmark. The two datasets may also respond differently because one is bipartite and the other is unipartite. The study is designed to distinguish these outcomes rather than to assume one in advance.

The intended outputs are:

1. a documented reproduction of TGF's random-injection experiments;
2. a configurable family of leakage-aware, structure- and time-aware generators;
3. diagnostics showing what each generator preserves and changes;
4. feature-reliance and cross-generator generalization analyses; and
5. , if warranted by the results, a small interpretable feature extension.

The study will not claim that any synthetic generator represents all real fraud or attacks. Its narrower claim concerns how benchmark construction affects conclusions about detectors. The required work is feasible because the selected paper provides code and example datasets, while the implementation focuses on a controlled injector replacement and an auditable evaluation wrapper. Optional micro-cluster or deep-learning baselines will be considered only after the required reproduction, diagnostics, and core experiments are complete.

4 Papers to Be Read

References

[1] Matthieu Latapy and Stephany Rajeh. *Trivial Graph Features and Classical Learning are Enough to Detect Random Anomalies.* IEEE International Conference on Data Mining (ICDM), pages 1330–1339, 2025. DOI: 10.1109/ICDM65498.2025.00142.
Primary paper selected for this project; provides the random-injection protocol, lightweight graph features, and Random Forest baseline.

[2] Siddharth Bhatia, Bryan Hooi, Minji Yoon, Kijung Shin, and Christos Faloutsos. *MIDAS: Microcluster-Based Detector of Anomalies in Edge Streams.* AAAI Conference on Artificial Intelligence, 2020.
Online detection of groups of suspicious edges; relevant to the optional coordinated-anomaly experiments.

[3] Dhivya Eswaran and Christos Faloutsos. *SedanSpot: Detecting Anomalies in Edge Streams.* IEEE International Conference on Data Mining (ICDM), 2018.
Detects anomalous streaming edges using burst information and graph connectivity; background on structural signals for edge anomalies.

[4] Dhivya Eswaran, Christos Faloutsos, Sudipto Guha, and Nina Mishra. *SpotLight: Detecting Anomalies in Streaming Graphs.* ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2018.
Streaming-graph anomalies such as unusual dense structures; a broader perspective on structured graph anomalies.

[5] Wenchao Yu, Wei Cheng, Charu C. Aggarwal, Kai Zhang, Haifeng Chen, and Wei Wang. *NetWalk: A Flexible Deep Embedding Approach for Anomaly Detection in Dynamic Networks.* ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2018.
Dynamically updated graph embeddings; shows how learned representations capture structural anomalies.

[6] Li Zheng, Zhenpeng Li, Jian Li, Zhao Li, and Jun Gao. *AddGraph: Anomaly Detection in Dynamic Graph Using Attention-Based Temporal GCN.* International Joint Conference on Artificial Intelligence (IJCAI), 2019.
Temporal graph neural network combining structure and time; a comparison point for feature-based approaches.

[7] Xiaoxiao Ma, Jia Wu, Shan Xue, Jian Yang, Chuan Zhou, Quan Z. Sheng, Hui Xiong, and Leman Akoglu. *A Comprehensive Survey on Graph Anomaly Detection With Deep Learning.* IEEE Transactions on Knowledge and Data Engineering, 35(12):12012–12038, 2023. DOI: 10.1109/TKDE.2021.3118815.
Taxonomy of problems, datasets, methods and evaluation practices; used for background and related work.